Module Five Assignment

Filip Arghir

Southern New Hampshire University

CS 499 Computer Science Capstone

Wasim Alim

6/5/2026

Module Five Assignment

For Milestone Four, I am still working with my CS 330 Computational Graphics and Visualization final project as my artifact for the Databases category. This project was originally created during CS 330 and is an OpenGL scene built in C++. The scene has two cars, the lamp post, sky as a background, a tree, a grass island, textures, lighting, materials, and camera interaction. I have continued focusing on this artifact as it aligns heavily with my career goals in becoming a technical artist, a programmer, and a game developer, and has allowed me to develop my skills further. I also wanted to focus on the same project across each milestone, so the final ePortfolio shows growth instead of three different projects.

I have focused on this artifact for my ePortfolio because I believe it shows visual and technical problem-solving. The project is not just a program that shows a scene or output but is a scene where code controls what is on the screen. This, I believe, makes it a great artifact to show my ability to work with rendering, transformations, textures, materials, data, and projects in general. For this milestone, I wanted to improve the project in terms of a database by moving scene object data outside of the C++ code and into a structured file.

Before this enhancement, the static scene objects were organized through a struct and vector from the Milestone Three enhancement. This greatly improved the project as the scene objects were stored as structured data and rendered through a loop. The actual object values were still written directly inside the C++ file. This means that if I were to adjust the sky, ground, tree, grass, island, or any part of the scene, I still had to edit the source code. For the database enhancement, I moved this static scene object into a data folder inside the project, specifically a CSV file named SceneObjects.csv.

The CSV file serves as a flat-file database. Each row in the file is one scene object, and each column is a specific field for that object. These fields include the object name, mesh type, scale, rotation, position, whether it's a color or a texture, whether it has a texture tag, a material tag, and more. This allows static scene objects to be stored as records rather than being manually adjusted and hardcoded into the render logic. The program then loads the CSV file at runtime and uses its data to populate the sceneObjects vector.

To properly work with this enhancement, I had to add a function to load scene objects from the CSV file. The loader opens the file, reads the rows and values, validates the data, and then creates a scene object from that row. If the row is missing information or has an invalid mesh, the program will skip that row and print a warning instead of crashing. I did want a fallback system, so if the CSV file is missing or has issues, the program will still use the default hardcoded scene objects. This keeps the project stable and allows the scene to use external data when the CSV is available.

This enhancement improves the artifact as it separates scene data from the rendering. Instead of needing to edit C++ every time one static object to be changed, the object values can be edited in a CSV file. This makes the project much easier to update and also makes the scene data easier to understand because it is organized in rows as well as columns. It also shows how database concepts can still apply to a graphics project without a full SQL database. The CSV acts as a small data source for the scene and is structured.

The main course outcome this enhancement supports would be the ability to use well-founded techniques, skills, and the proper tools in computing practices to implement solutions that have value. This enhancement adds value because it makes the project more flexible and easier to work with and maintain. It also supports the outcome related to professional

communication because I can explain how the scene data is structured, how the loader works, and why moving data outside of the source code improves the project. It also connects to the security aspect as the loader validates data and handles missing or incorrect files through warning and fallback behavior instead of allowing the program to fail unexpectedly.

The biggest challenge was making the database enhancement meaningful without developing a separate project or being overly complicated. I didn't want to add a SQL or MongoDB database, or a larger external database, as that would have increased my scope too much and caused build issues inside of the original project. Instead, using a CSV file was a realistic, simpler choice for the project and still demonstrates database concepts such as records, fields, structured data validation, and separates data from logic. It also keeps the enhancement manageable and fairly simple to explain.

One other challenge was making sure the scene still looked the same after moving the data into the CSV file. The object values had to match the original static scene objects, so the visual output wouldn't change. I verified that the CSV file also loaded correctly and checked the console output, which showed that the program loaded eight objects from the CSV file. This then confirmed that the scene was using the external data file instead of only relying on the fallback object list.

Overall, this milestone was a great experience. I've worked with CSV files previously, and applying those database concepts not only in traditional web applications or business systems, but I also felt clever applying it in a graphical project. The scene still looks the same, but objects are stored and loaded better. This makes the artifact even stronger for my ePortfolio as it now demonstrates software design, algorithms, data structures, as well as database organization, all within the same project.